ETHEREUM PEERDAS · CODING SCHEME COMPARISON

Benchmarking 1D-RS, 2D-RS and Block-Circulant Codes

Seven parameterisations, one blob, 520 trials

Benchmarking 1D-RS, 2D-RS, and Block-Circulant codes

A controlled comparison of erasure-coding schemes as candidates for Ethereum's data-availability sampling: one-dimensional Reed-Solomon (what PeerDAS uses today), two-dimensional Reed-Solomon, and Block-Circulant (BC) codes at five parameterisations. All of them run over the same field, on the same blob, under the same erasure model, and are judged by the same success criterion.

The question each scheme has to answer is the same: **if a fraction of the encoded data goes missing, can the original blob still be reconstructed exactly — and what did that cost in storage and in decode time?**

This revision changes two things, both following supervisor guidance of 2026-09-15 and both aimed at making the comparison fair rather than at making any scheme look better.

2D-RS moves from an 88×88 grid to 88×96, putting its rate at 0.485 against the others' 0.500 — the closest any grid reachable in this field gets to 0.5, and a correction of the previous 0.529. The natural-looking 90×90 turns out to be unbuildable here, for a reason worth stating up front because it is easy to miss: a length-n NTT needs n to divide $p-1$, and $90 = 2{\cdot}3^2{\cdot}5$ needs both a 3^2 (this field's $p-1$ carries only 3^1) and a 5 (absent entirely).

The BC rows move from $\lambda = \mu$ to $\lambda \in \{2, 3\}$, with one $\lambda = 4$ row kept as an anchor. The previous set swept $\lambda = \mu$ at $\mu = 4, 8, 16$, which measured the same extreme three times and pushed the decoder's reach down to 0.059. $\lambda = 16$ is not implementable, and the source paper's algorithms assume $\lambda = 2$, so the sweep now sits where the construction actually lives. This turns out to matter a great deal: the best BC row now tolerates 0.35 rather than 0.20.

The practical effect of both changes is that every scheme in the table below stores within 3% of twice its blob, so differences in what they tolerate are differences between the codes and their decoders, not between their budgets.

How the code works

The whole experiment is five moving parts. Each is a separate file, and the benchmark drives them without reimplementing any of their logic.

Packing a blob (`blob_generator.py`). An Ethereum blob is 4096 field elements of the BLS12-381 scalar field. Each element is 32 bytes on the wire but only 31 are usable, because the value must stay below the field modulus — so one blob carries 126,976 bytes of application data. This is the unit everything else operates on: a list of 4096 field elements, and a function to turn it back into the original bytes.

1D-RS (`1D_RS_Galois.py`). The key idea is that the blob's 4096 elements are not raw data to be copied — they are the *values of a single polynomial* sampled at 4096 fixed points. Encoding recovers that polynomial (an inverse NTT turns 4096 values into 4096 coefficients), pads it with zeros, and evaluates it again at 8192 points (a forward NTT). The result is 8192 symbols describing the same polynomial. Since a polynomial of degree below 4096 is pinned down by *any* 4096 of its values, any 4096 surviving symbols reconstruct everything. Decoding builds an erasure-locator polynomial whose roots sit exactly at the missing positions, multiplies through by it, inverts the transform, and divides the locator back out — recovering the original polynomial in one pass. Crucially it first checks whether the number of erasures exceeds `n−k`; if so it gives up immediately rather than attempting anything.

2D-RS (`2D_RS_Galois.py`). The same interpolate-and-extend step, applied twice. The blob is reshaped into a 64×64 grid; every row is extended, then every column of that result is extended. Because the original grid is a single bivariate polynomial, every row *and* every column of the result is independently a valid 1D-RS codeword. Decoding exploits this: it sweeps all columns, then all rows, repeatedly. A line holding too many erasures to fix alone often becomes fixable once the perpendicular sweep has repaired some of its cells, so repairs cascade across the two axes until nothing more can be done.

What changed here is the extension factor, twice. A product code's rate is the *product* of its two component rates, so doubling both axes — the original 64→128 setting — gives rate 1/4 and 4× storage. That made the first comparison unfair: 2D-RS was being judged at the same erasure rate as codes spending half as much. The fix is to pick the component rate so that its square lands on 0.5: each axis should extend 64 → 64/√0.5 = 90.5, which is where the "64 × √2 ≈ 90, so use 90×90" suggestion comes from. The reasoning is right; the number is unavailable.

A length-n NTT exists only if n divides $p-1$, and for the BLS12-381 scalar field

> $p - 1 = 2^{32} \cdot 3 \cdot 11 \cdot 19 \cdot 10177 \cdot 125527 \cdot 859267 \cdot 906349^2 \cdot 2508409 \cdot 2529403 \cdot \ldots$

$90 = 2 \cdot 3^2 \cdot 5$ needs a 3^2 where only 3^1 is available, and a 5 that does not appear at all. There is no length-90 NTT here, so no 90×90 grid. The reachable lengths above 64 are 66, 76, 88, 96, 114, 128, 132, … — and 88 and 96 bracket the 90.5 target from either side, with geometric mean

$\sqrt{8448} = 91.9$ sitting close to it. Component codes need not be equal, so the grid is now **88 × 96**: component rates $64/96 = 0.667$ and $64/88 = 0.727$, overall $4096/8448 =$ **0.485** at 2.06× storage. That is the closest to 0.5 of any reachable pair. The previous 88×88 gave 0.529 at 1.89×, further off on both counts *and* cheaper than the codes it was compared against.

Exactly 0.5 is unreachable regardless: it would need the codeword to be 8192 cells, a power of two forces both grid dimensions to be powers of two, and each axis must at least double to carry any parity at all — which lands on 16384, not 8192.

Block-Circulant (`BCcodeEthereum.py`). Structurally different: instead of one global polynomial, the message is cut into μ small blocks of ω symbols, and each block gets ρ parity symbols computed from its own information plus that of λ consecutive neighbours, wrapped around a ring. Overhead is 2×, the same as 1D-RS.

The parameterisation matters more than it first appears, and λ turns out to be the parameter that matters. Two quantities move in *opposite* directions as it changes:

- The **code's** guarantee. Minimum distance is $d = \lambda\rho + 1$, so the fraction of erasures it survives against *any* pattern is $(d-1)/n = \lambda\rho/n$. This **rises** with λ.
- The **decoder's** reach. `DecCBC[2]` resolves a local block when the erasures landing in its support of $N_0 = \lambda\omega + \rho$ symbols number at most ρ, i.e. while the erasure rate is under $\rho/N_0 = 1/(1+\lambda)$ when $\rho = \omega$. This **falls** with λ.

So a larger λ buys a better code and a worse decoder. $\lambda = \mu$ is the extreme: d reaches $4097 = n - k + 1$, the Singleton bound, making the code MDS — exactly as good as Reed-Solomon — while the decoder's reach bottoms out at $1/(1+\mu)$.

That is what the previous revision swept, at $\mu = 4, 8, 16$: three MDS codes whose decoders reached 0.200, 0.111 and 0.059. It measured the same extreme three times and never sampled the regime the construction is for. Supervisor guidance is that `DecCBC` is practical only to $\lambda = 3$, that $\lambda = 16$ is not implementable, and that analysis belongs at $\lambda = 2$ and 3 — which is also where the source paper sits, as the next section shows. The rows swept here follow that, keeping one $\lambda = 4$ row as the MDS anchor.

Note what that produces: **ω2048 λ2 and ω1024 λ4 are both MDS with $d = 4097$ — identical as codes — yet their decoders reach 0.34 and 0.20.** Same code quality, different reach, from λ alone. Separating the code from the decoder is what this sweep measures, and that pair is the cleanest statement of it.

Checking the parameters against the source

The BC construction comes from Sasidharan, Viterbo and Feist, *Block Circulant Codes for Ethereum PeerDAS* (ePrint 2026/708). Its notation table matches this implementation exactly — $[N_0, K_0] = [\lambda\omega + \rho, \lambda\omega]$, $[n, k] = [\mu(\omega+\rho), \mu\omega]$, $R = \omega/(\omega+\rho)$, $S = \lambda\rho/n$, $S_0 = \rho/N_0$ — which is worth

recording, because two of those are the quantities this report now reports and one of them was derived here independently before the paper was consulted:

- The paper's **stopping rate** $S = \lambda\rho/n$ is precisely the $(d-1)/n$ erasure tolerance used throughout below.
- The paper's **local stopping rate** $S_0 = \rho/N_0$ is precisely the decoder reach predicted above. The prediction and the paper's definition are the same quantity, arrived at separately.

Two further points from it bear on the rows chosen here. First, the paper **fixes λ = 2** for its encoding and reconstruction algorithms and for its entire performance analysis, and states the λρ erasure-correction result for **λ = 2, 3 only**. The λ = 2 rows therefore sit exactly where the paper does, the λ = 3 row sits at the edge of its stated range, and the λ = 4 row is outside it — its $d = 4097$ extrapolates a formula past where it was proved, which is another reason to read it as an anchor rather than a result.

Second, the paper's worked example is μ = 4, λ = 2, ρ = ω with **ω = 32 cells at stacking depth *D* = 64** — it counts ω and ρ in *cells*, not in field symbols. Converting: a 4096-symbol blob at $D = 64$ is 64 cells, so μ = 4 gives ω = 16 cells = **1024 field symbols**. The ω1024 λ2 row below *is* the paper's worked configuration, expressed in symbols.

The harness (`benchmark_1D_2D_BC.py`). Each scheme is wrapped in a small adapter exposing the same five operations — `encode`, `random_mask`, `corrupt`, `decode`, `extract_message` — so the sweep loop never needs to know that 2D-RS's codeword is a grid while BC's is a flat vector. The adapters call the real functions in the files above; no coding logic is duplicated, so fixing one of those files changes the benchmark's results automatically.

The experimental pipeline

Every measurement follows the same seven steps, and only two of them differ between schemes:

1. **Generate** 126,976 random application bytes.
2. **Pack** them into one blob of 4096 field elements, the real EIP-4844 way.
3. **Encode** — *scheme-specific* — expanding 4096 elements into a codeword of `n` elements.
4. **Erase** — withhold each codeword position independently with probability equal to the target rate, simulating samples that peers failed to provide.
5. **Decode** — *scheme-specific* — attempt to reconstruct the full codeword from what survived.
6. **Extract** the original 4096 elements back out of the recovered codeword.
7. **Verify** by unpacking to bytes and comparing against step 1.

Steps 1, 2, 4, 6 and 7 are byte-identical across every scheme. That is what makes the comparison fair: any difference in the results has to come from step 3 or step 5, which is precisely what is being compared.

Step 7 is the one worth dwelling on. Success is defined as *the recovered bytes equal the original bytes* — not as "the decoder reported success", and not even as "the field elements matched". That distinction is not pedantry: one of these decoders turned out to report success on data it had silently corrupted, and only the byte comparison caught it.

What is held identical

- **Field** — the real BLS12-381 scalar field.
- **Message** — the same 4096 elements, packed from the same bytes, fed to every encoder. Not one random blob per scheme.
- **Erasure model** — independent per-position withholding at a shared target rate. 1D-RS and all five BC rows share `n = 8192`, so on each trial they are corrupted at the *same literal positions*, not merely the same rate. 2D-RS has `n = 8448` and draws its own masks at the same rate.
- **Success criterion** — the full byte-level round trip described above.

Storage is matched to within 3%: 2.00× for 1D-RS and every BC row, 2.06× for 2D-RS. The small residual difference is reported rather than controlled away, and it runs *against* 2D-RS rather than for it.

Method

Erasure rates are swept from 2% to 55%. Trial counts vary by scheme and rate: a scheme decoding against a formal budget check behaves deterministically well away from its threshold, so one trial there is representative, while near a threshold the outcome genuinely varies from draw to draw and needs several. Trials are therefore concentrated around each scheme's own transition — which is not the same rate for all of them. Each success rate carries a 95% Wilson confidence interval.

Code parameters and storage cost

All of them carry the same 124 KiB blob payload; they differ in how much they store to protect it. `R = k/n` is the code rate, and `1−R` is the largest erasure fraction *any* code of that rate could survive. The two rows the supervisor asked for — **N_0 and K_0, the component (local) code length and dimension** — are included: for 1D-RS the global code is the only code, for 2D-RS the component is one axis (the row axis is shown; the column axis is [88, 64]), and for BC it is one local block.

Note the gap between the last two rows for 2D-RS. Its minimum distance is the *product* of its component distances, $33 \times 25 = 825$, so its guarantee is 0.098 — nowhere near the 0.515 its rate would permit. A product code is never MDS; that is the structural price of two-axis locality.

Comparing 2D-RS against $1-R$, as the previous revision did, measured it against a bound its construction cannot reach.

	1D-RS	2D-RS	BC ω2048 λ2	BC ω1024 λ2	BC ω512 λ2	BC ω1024 λ3	BC ω1024 λ4
Message k	4096	4096	4096	4096	4096	4096	4096
Codeword n	8192	8448	8192	8192	8192	8192	8192
Code rate R	0.500	0.485	0.500	0.500	0.500	0.500	0.500
Storage overhead	2.00×	2.06×	2.00×	2.00×	2.00×	2.00×	2.00×
Stored per blob	256 KiB	264 KiB	256 KiB	256 KiB	256 KiB	256 KiB	256 KiB
Component $\mathbf{N}_0$	8192	96	6144	3072	1536	4096	5120
Component $\mathbf{K}_0$	4096	64	4096	2048	1024	3072	4096
Min. distance d	4097	825	4097	2049	1025	3073	4097
Guarantee $(d-1)/n$	0.500	0.098	0.500	0.250	0.125	0.375	0.500
Rate bound $1-R$	0.500	0.515	0.500	0.500	0.500	0.500	0.500

The five Block-Circulant rows differ in how the same 4096 message symbols are cut into blocks and in how far the blocks overlap. Every row sets $\rho = \omega$, which fixes $n = 2k = 8192$ and rate 0.5 throughout, so λ and μ are the only things varying. Read the last three rows together: the *predicted* decoder reach S_0 and what was *measured* agree at every point, and both are governed by λ rather than by ω or μ.

	BC ω2048 λ2	BC ω1024 λ2	BC ω512 λ2	BC ω1024 λ3	BC ω1024 λ4
Information/block ω	2048	1024	512	1024	1024
Parity/block ρ	2048	1024	512	1024	1024
Local blocks μ	2	4	8	4	4
Overlap λ	2	2	2	3	4
Local length N_0	6144	3072	1536	4096	5120
Local dim K_0	4096	2048	1024	3072	4096
Local rate R_0	0.667	0.667	0.667	0.750	0.800
Min. distance *d*	4097	2049	1025	3073	4097
Stopping rate *S*	0.500	0.250	0.125	0.375	0.500
Predicted $S_0 = \rho/N_0$	0.333	0.333	0.333	0.250	0.200
Measured	**0.34**	**0.35**	**0.35**	**0.28**	**0.20**
Local solve size	2048×2048	1024×1024	512×512	1024×1024	1024×1024

Encode and decode cost, from the same run. Decode times are the mean over trials that actually recovered the blob; a dash means nothing recovered at that rate, so there is no decode time to report.

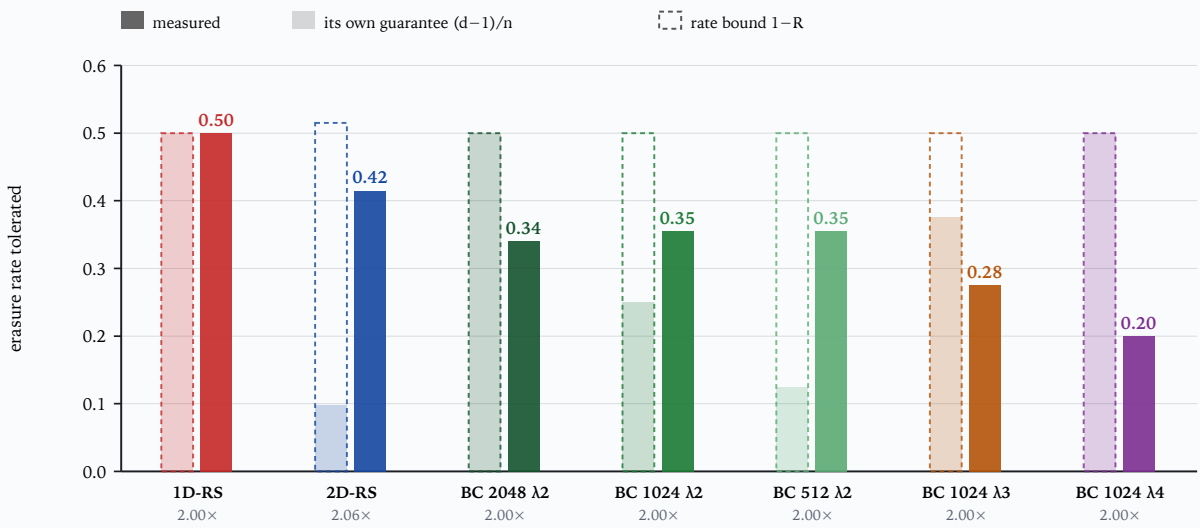
	1D-RS	2D-RS	BC ω2048 λ2	BC ω1024 λ2	BC ω512 λ2	BC ω1024 λ3	BC ω1024 λ4
Encode	0.46s	2.65s	14.16s	6.65s	3.56s	7.86s	9.00s
Decode @ 2%	5.18s	2.44s	0.41s	0.24s	0.15s	0.30s	0.38s
Decode @ 5%	11.10s	3.11s	0.98s	0.54s	0.32s	0.69s	0.85s
Decode @ 10%	20.55s	3.35s	2.01s	1.09s	0.63s	1.39s	1.70s
Decode @ 20%	40.36s	5.08s	4.43s	2.33s	1.32s	2.96s	3.61s
Decode @ 30%	56.94s	4.50s	7.15s	3.71s	2.08s	—	—
Decode @ 35%	63.25s	5.85s	—	4.30s	2.43s	—	—
Decode @ 48%	81.77s	—	—	—	—	—	—

Results

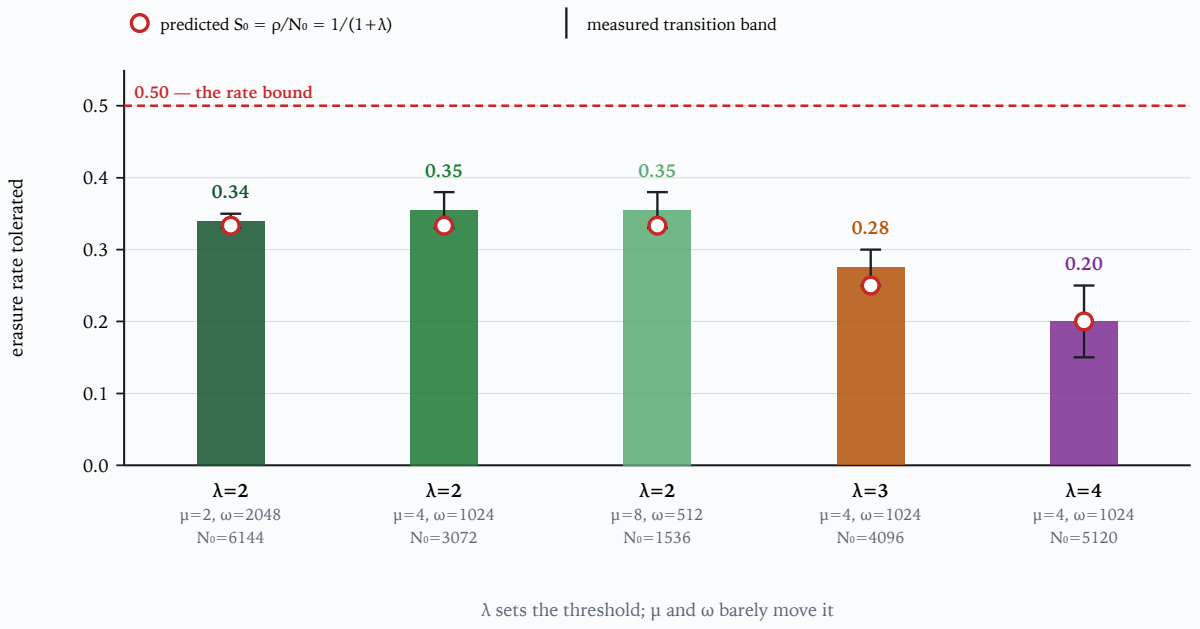
Recovered trials at each erasure rate. Trial counts differ by scheme and by rate — more where a scheme is near its own threshold and the outcome genuinely varies, fewer where it does not — so each cell shows recovered/attempted rather than a bare percentage.

erasure rate	1D-RS	2D-RS	BC ω2048 λ2	BC ω1024 λ2	BC ω512 λ2	BC ω1024 λ3	BC ω1024 λ4
2%	1/1	3/3	4/4	4/4	4/4	4/4	4/4
5%	1/1	3/3	4/4	4/4	4/4	4/4	4/4
8%	1/1	3/3	4/4	4/4	4/4	4/4	4/4
10%	1/1	3/3	4/4	4/4	4/4	4/4	4/4
12%	1/1	3/3	4/4	4/4	4/4	4/4	4/4
15%	1/1	3/3	4/4	4/4	4/4	4/4	8/8
20%	1/1	3/3	4/4	4/4	4/4	8/8	5/8
25%	1/1	3/3	4/4	4/4	4/4	8/8	0/8
30%	1/1	3/3	8/8	8/8	8/8	0/8	0/4
33%	1/1	3/3	8/8	8/8	8/8	0/4	0/4
35%	1/1	3/3	0/8	1/8	4/8	0/4	0/4
38%	2/2	5/5	0/8	0/8	0/8	0/4	0/4
40%	2/2	2/5	0/4	0/4	0/4	0/4	0/4
45%	2/2	0/5	0/4	0/4	0/4	0/4	0/4
48%	3/3	0/3	0/4	0/4	0/4	0/4	0/4
50%	2/3	0/3	0/4	0/4	0/4	0/4	0/4
52%	0/3	0/3	0/4	0/4	0/4	0/4	0/4
55%	0/2	0/3	0/4	0/4	0/4	0/4	0/4

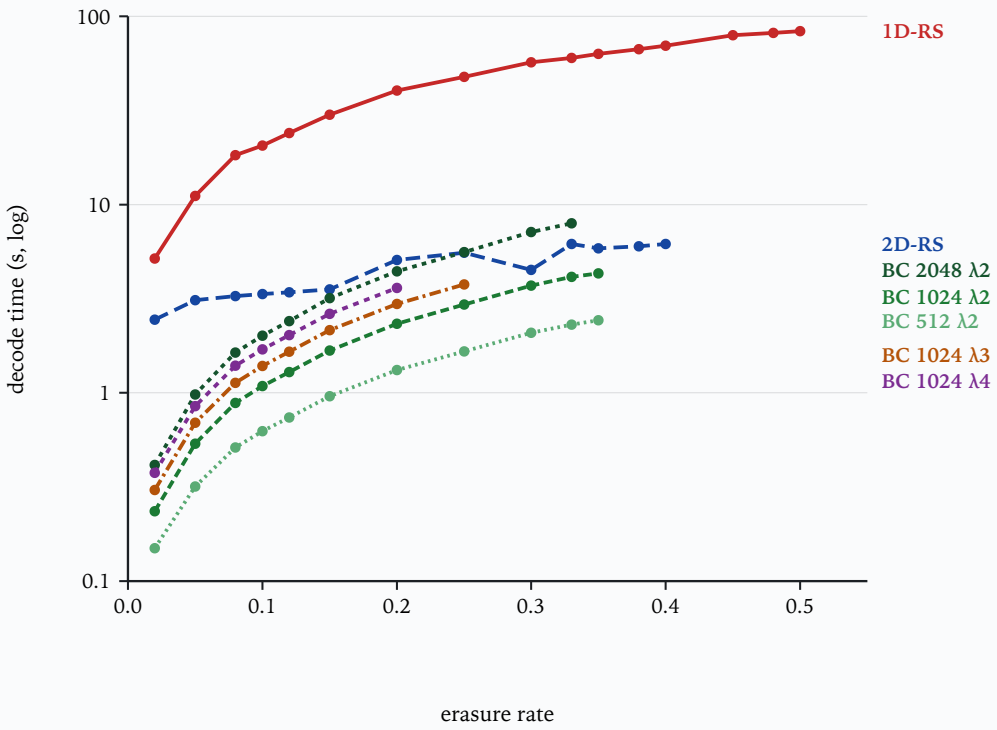
Across all 520 trials, no scheme returned corrupted data while reporting success: silent corruption was zero everywhere (0/28 for 1D-RS, 0/60 for 2D-RS, and 0/88, 0/88, 0/88, 0/84, 0/84 across the five BC rows). Before the decoder fix described below, BC's figure was 31 of 40.



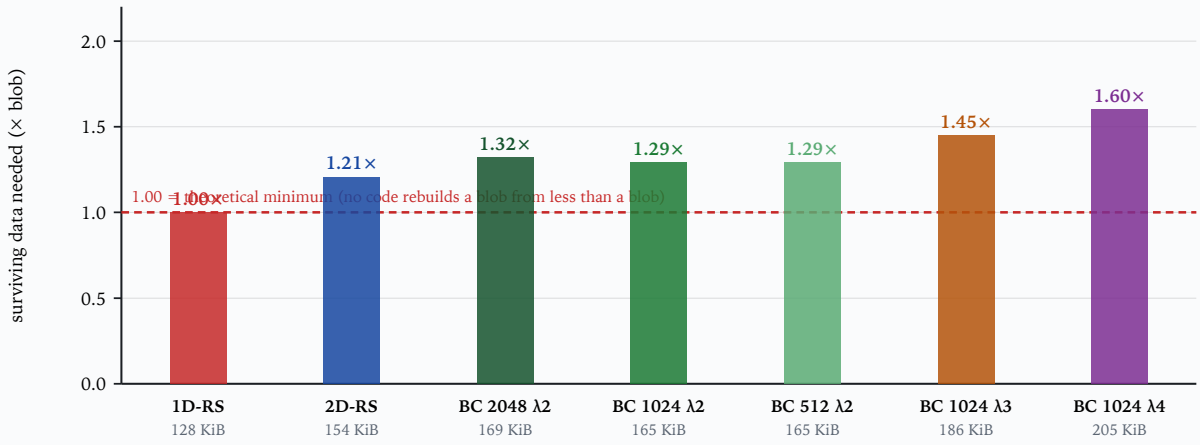
What each scheme actually tolerated (solid), beside its own minimum-distance guarantee (pale) and the rate bound $1-R$ that no code of that rate can beat (dashed). Storage is matched to within 3%, so the bars read against each other directly. 1D-RS lands exactly on its bound — that is what being MDS means. Three schemes sit *above* their own guarantee: 2D-RS at more than four times it, and both λ=2 block-circulant rows with μ>2. That is not a contradiction — a guarantee holds against every pattern including the adversarial worst case, while these trials draw patterns at random. Solid bars are the midpoint of the measured transition.



The block-circulant parameter sweep, and the clearest single result here. The decoder's reach is set by λ alone: all three λ=2 rows land together at 0.34–0.35 despite μ ranging over 2, 4, 8 and ω over 2048, 1024, 512 — a factor of four in block size that moves the threshold by nothing. Raising λ to 3 drops it to 0.28, and to 4 drops it to 0.20. The open markers are the prediction $S_0 = \rho/N_0 = 1/(1+\lambda)$, which is the local code's stopping rate in the source paper's own notation; the bars are what was measured, and the two agree at every point.



Mean decode time of the trials that actually recovered, against erasure rate, log scale. Failed trials are excluded: above its threshold a scheme rejects the pattern on a budget check in under a millisecond having attempted nothing, which is not a decode time. 1D-RS is an order of magnitude more expensive than everything else at every rate, because it divides one 8192-length polynomial while the others divide short ones. Every block-circulant row is cheaper than 1D-RS at the same rate, and cheaper than 2D-RS too below about 30%.



How much surviving data each scheme needs to rebuild one blob, as a multiple of the blob itself. This is the question that matters for data-availability sampling, where a node has to actually fetch those symbols. 1D-RS needs essentially the theoretical floor — no code rebuilds 124 KiB from less than 124 KiB. Every other scheme needs more, and a scheme tolerating a lower erasure rate at the same storage cost necessarily needs more.

What the results show

1D-RS behaves exactly as Reed-Solomon theory says it must. It recovers everything up to 48% and nothing from 50% upward. The invariant is exact rather than approximate: across every 1D-RS trial in the sweep, recovery succeeded if and only if the drawn erasure count was at most $n-k = 4096$, with no exceptions. The partial results at the threshold are not the decoder being unreliable — they are the random mask landing on either side of a hard line. Being MDS, 1D-RS extracts essentially all of its theoretical budget.

2D-RS comes second, and beats its own guarantee by a wide margin. It holds to 35% and fails from 45%, with the transition straddling 38–40% (5/5 at 38%, 2/5 at 40%, 0/5 at 45%) — call it 0.42. Two reference points matter and they are very far apart. Its rate would permit 0.515; its *minimum distance* only guarantees 0.098, because a product code's distance is the product of its component distances, $33 \times 25 = 825$. The measured 0.42 is more than four times that guarantee.

That gap is the cross-axis cascade doing real work. A guarantee has to hold against the worst pattern in existence — for a product code, an adversary who concentrates erasures to kill whole rows and columns at once. Random patterns are nothing like that, so lines that are individually over budget keep getting rescued once their partner axis repairs enough neighbours. The honest summary is that 2D-RS's *typical* behaviour is far better than its *guaranteed* behaviour, and which of the two matters depends entirely on whether the erasures are adversarial. Against the $1-R$ bound it reaches 82%; against its own distance it is at 430%.

The Block-Circulant rows separate the code from the decoder, and λ is what moves the decoder. This is the clearest result in the sweep:

- **ω2048 λ2** (μ=2, MDS, d=4097): recovers to 33%, fails from 35%; predicted $S_0 = 0.333$.
- **ω1024 λ2** (μ=4, d=2049): recovers to 33%, fails from 38%; predicted $S_0 = 0.333$.
- **ω512 λ2** (μ=8, d=1025): recovers to 33%, fails from 38%; predicted $S_0 = 0.333$.
- **ω1024 λ3** (μ=4, d=3073): recovers to 25%, fails from 30%; predicted $S_0 = 0.250$.
- **ω1024 λ4** (μ=4, MDS, d=4097): recovers to 15%, fails from 25%; predicted $S_0 = 0.200$.

Two things fall out of that list. First, **the prediction is right every time** — $S_0 = \rho/N_0 = 1/(1+\lambda)$, the source paper's local stopping rate, lands inside the measured transition band for all five rows. The limit is not mysterious; it is arithmetic about how many erasures fit inside one local support.

Second, **λ is the only parameter that moves it.** The three λ=2 rows all land at 0.34–0.35 while μ ranges over 2, 4, 8 and ω over 2048, 1024, 512 — a factor of four in block size that changes the threshold by nothing at all. Raising λ to 3 costs 7 points, and to 4 costs 14. This is the direct answer to "which knob should we turn": not ω, not μ, but λ, and downward.

Third, and most useful for the construction's prospects, **two of these rows decode past their own worst-case guarantee.** ω1024 λ2 is guaranteed only 0.250 and reaches 0.35; ω512 λ2 is guaranteed 0.125 and also reaches 0.35, nearly three times its bound. These are *not* MDS codes

and cannot claim to be; against a worst-case adversary they would fail where the table says they fail. But under random erasure — which is what a sampling network actually produces absent a targeted attack — they perform like codes with far better distance. That is the sense in which parameters that are "not fully decodable" still demonstrate the construction's potential.

Set against that, the $\lambda=2$ rows cost less decode time than anything except each other, and every BC row is cheaper than 1D-RS at every rate measured. The local system each solves is $\rho\times\rho$, which generic elimination handles in ρ^3 — which is why the pairwise phase stays unaffordable while the single-block phase does not, the latter's system being Vandermonde and so dropping to ρ^2.

Making these parameterisations runnable

The larger-ω BC rows could not be measured at all with the decoder as it stood. The reason is worth recording, because it is the same wall anyone re-running this will hit.

The original `gf_solve` is a Gauss-Jordan elimination written as a Python loop over rows. Its cost grows as the cube of the matrix dimension, and this field's arithmetic runs in `galois`' object mode at roughly 1.3 microseconds per element operation. At $\omega=\rho=2$ the local systems are 2×2 and none of that matters. At $\omega=\rho=1024$ they are 1024×1024, which is about a billion element operations — roughly 25 minutes for a single local solve, with μ of them per decoding round.

Three changes brought it into range, none of which alters what is computed:

The elimination step is now one broadcast update instead of a loop over rows. Same arithmetic, same element count, but the per-pivot dispatches into object-mode arithmetic drop from m to one. Measured: 1.34 → 0.55 microseconds per element operation, a 2.4× speedup.

The single-block solve now exploits the structure it always had. The local parity-check matrix is a GRS matrix, `H[r, c] = (c+1)^r`. Restricted to the erased columns and to its first e rows it is a square Vandermonde in distinct evaluation points — so it is always invertible, and it can be solved in $O(e^2)$ by a node-polynomial argument rather than $O(e^3)$ by elimination. Measured at $e = 1024$: 0.87 seconds against an extrapolated 1441, about 1650×. This is what makes $\omega=1024$ tractable.

The global parity-check matrix is no longer built. It is $(\mu\rho)\times n = 4096\times8192 = 33.5$ million field elements, and every row the decoder needs can be regenerated from (ρ, n_0) on demand.

A useful side effect: because the single-block system is provably square Vandermonde in distinct nodes, it cannot be underdetermined, so the class of bug described below cannot arise on that path at all.

The pairwise phase does not get a turn

`DecCBC[2]`'s second phase stacks two adjacent blocks' parity checks and solves them jointly. That system is *not* Vandermonde — the two row groups assign different evaluation points to the same shared columns — so it cannot use the fast solver and needs full elimination, at cubic cost.

Phase 2 only runs when phase 1 has stalled, which happens only once a local support carries more than ρ erasures. Every row swept here has $\rho \geq 256$, so every pair that could possibly help carries more than 256 unknowns. The sweep applies a stated compute budget of 256 unknowns per pairwise system, identically to all five rows, and above that budget a pair is skipped, counted, and its positions left honestly flagged.

The consequence is that **all five rows are measured as single-phase decoders**, and this is a limit of affordable computation rather than of the code. The cost was measured directly on the 512-equation systems that $\omega=\rho=256$ produces:

unknowns in the pair	erasure rate it corresponds to	per pair	per decoding round (16 pairs)
276	6%	21.7s	5.8 min
369	8%	39.5s	10.5 min
461	10%	61.7s	16.4 min

That is for the *smallest* local block swept here, and one decoding round is not one decode — the loop repeats while it makes progress. At $\omega=\rho=1024$ a full pair is 2048 equations in 2048 unknowns; at the 0.57 microseconds per field operation these timings imply, that is about 80 minutes for a single pair. Lifting the cap enough to change any result would take days per sweep.

The honest reading of the BC numbers is therefore: this is what the single-block phase of `DecCBC[2]` achieves. The pairwise phase would roughly double the budget from ρ to 2ρ, which the same arithmetic puts at 0.333 for $\omega=1024$, 0.200 for $\omega=512$ and 0.111 for $\omega=256$ — still short of the 0.50 these codes permit.

The Block-Circulant decoder fix

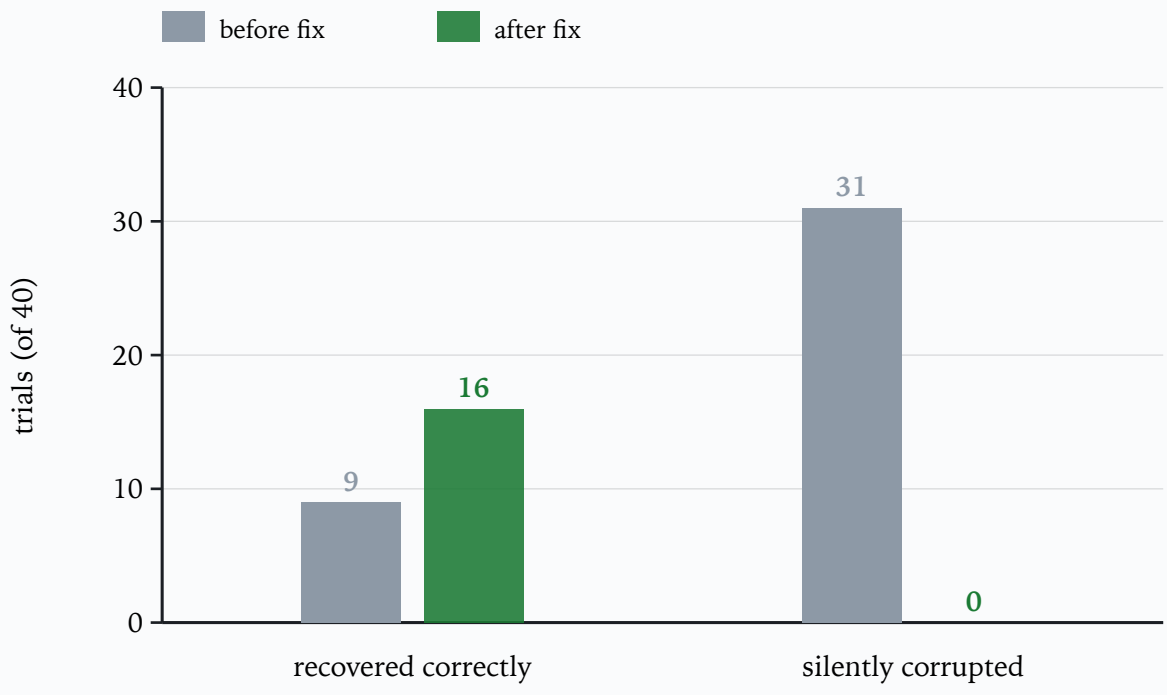
The BC results are from a corrected decoder. The version originally benchmarked had two faults that together made its output untrustworthy rather than merely weak. Both were found at the earlier $\omega=\rho=\lambda=2$ parameterisation, and the before/after figure below is measured there.

`gf_solve` fabricated values. The Gaussian elimination routine returned zero for any unknown whose column never received a pivot. To the caller that is indistinguishable from having genuinely solved for zero, so an underdetermined system silently produced a confident-looking wrong an-

swer. It now raises `GFSolveError` unless the system has exactly one solution — every column pivoted, and no row reduced to `0 = nonzero`.

`dec_cbc_2`'s pairwise phase accepted systems it had not solved. Its guard is `0 < erasures ≤ 2ρ`, which is necessary but not sufficient. The equations it stacks are block *i*'s own ρ parity checks on top of block *i+1*'s — not the auxiliary `[2(ω+ρ), 2ω, 2ρ+1]` GRS code over the union of the two supports that the source paper's Algorithm 1 constructs. The stacked form is not MDS, so a split such as "three erasures inside block *i*'s support, one inside *i+1*'s" leaves two equations facing three unknowns. Combined with the `gf_solve` fault, the decoder wrote zeros into those positions *and cleared their erasure flags*, reporting success on corrupted data. It now skips any pair it cannot uniquely solve, leaving those positions honestly flagged.

The practical effect is stark. Measured before the fix, across 40 trials spanning 5–45% erasure, `dec_cbc_2` claimed success **every single time** and was wrong on 31 of them; it never once reported a failure. Its `remaining` flag — the signal a caller would naturally trust — carried no information about whether the data was correct. Had this benchmark trusted that flag instead of checking bytes, it would have reported BC at 100% success at every rate.



The decoder fix, measured on the parameterisation it was found at (ω=ρ=λ=2, μ=2048) — *not* the rows swept above. The same 40 trials (5%, 15%, 25% and 45% erasure, ten each) run through the decoder before and after. Correct recoveries rose from 9 to 16, because the old decoder poisoned positions with fabricated zeros that blocked later rounds from resolving them. Silent corruptions — cases where the decoder reported success on wrong data — fell from 31 to zero. The remaining failures are honest: the decoder reports the positions it could not resolve.

A third change is a performance fix rather than a correctness one: the pairwise phase used to `break` and restart the single-block phase after every pair it repaired, paying a full scan of all local blocks per repair. It now continues through the remaining pairs in the same pass.

What none of this does is restore the correction radius the paper claims. Implementing the auxiliary GRS construction would let the pairwise phase handle *any* $\leq 2\rho$ erasures across two adjacent blocks, rather than only those solvable from the stacked parity checks — and, being a GRS code, it would also be solvable in $O(e^2)$ rather than $O(e^3)$, which would remove the compute wall described above. That is the single highest-value piece of remaining work on this file.

Reproducing

```
pip install galois numpy
python3 benchmark_1D_2D_BC.py
```

Per-trial results — erasure counts, decode times, success flags, and the decoder's own claim separately from ground truth — are written to `benchmark_results.json`.

Generated from the project repository's `docs/` source Markdown.